Payify Kart Mutabakat Sistemi — Derin Teknik Analiz Dokümanı

> **Versiyon:** 3.0
> **Son Güncelleme:** 15 Nisan 2026
> **Yazar:** Kıdemli Backend Analiz Uzmanı
> **Kapsam:** Tüm özelleştirilmiş endpoint'lerin kod okuma + davranış çıkarma + iş anlamı yorumlama analizi

---

İçindekiler

1. Dosya Yükleme (File Ingestion) Endpoint Analizi
2. Mutabakat Değerlendirme (Evaluate) Endpoint Analizi
3. Mutabakat Yürütme (Execute) Endpoint Analizi
4. Manuel Onay (Approve) Endpoint Analizi
5. Manuel Red (Reject) Endpoint Analizi
6. Bekleyen İncelemeler (Pending Reviews) Endpoint Analizi
7. Uyarılar (Alerts) Endpoint Analizi
8. Arşiv Önizleme (Archive Preview) Endpoint Analizi
9. Arşiv Çalıştır (Archive Run) Endpoint Analizi
10. Raporlama Endpoint Analizleri
11. Status ve State Geçiş Analizi
12. İş Kuralları Özeti
13. Veri Akışları
14. SQL View Okuma Rehberi
15. Rapor Yorumlama Kılavuzu
16. Analist Kılavuzu
17. Geliştirici Notları
18. Senaryo Bazlı Davranışlar

---

1. Dosya Yükleme (File Ingestion) Endpoint Analizi

1.1 Endpoint Amacı

`POST /v1/FileIngestion` ve `POST /v1/FileIngestion/{fileSourceType}/{fileType}/{fileContentType}`

Kart ağlarından (BKM, Visa, Mastercard) gelen işlem dosyalarını ve takas/mutabakat (clearing) dosyalarını sisteme alır. Dosyayı satır satır parse eder, veritabanına yazar, duplikat tespiti yapar, clearing eşleştirmesi uygular ve dosyayı hedef depoya arşivler. Bu endpoint tüm mutabakat sürecinin **başlangıç noktası**dır — dosya sisteme alınmadan hiçbir mutabakat operasyonu başlayamaz.

1.2 Request Alan Analizi

`FileSourceType` **(enum, zorunlu)**

Değer	Teknik Anlam	İş Anlamı	Davranış Etkisi

`Remote (1)`	FTP/SFTP üzerinden dosya listele ve oku	Kart ağından otomatik dosya çekme	Sistem konfigürasyondaki remote profil ayarlarını kullanarak dosyaları keşfeder
`Local (2)`	Yerel dosya sisteminden oku	Manuel yükleme veya test senaryoları	`FilePath` boşsa konfigürasyondaki varsayılan yerel dizinden okur; doluysa direkt o path'ten okur

- **Null/boş gelirse:** Request body tamamen null gelirse `INVALID_REQUEST` hatası döner. Enum default değeri (0) gelirse konfigürasyon bulunamaz ve `CONFIGURATION` step'inde hata oluşur.
- **Yanlış değer:** Tanımlı olmayan bir enum değeri gelirse `FlexibleEnumJsonConverter` deserialize sırasında hata fırlatır.
- **İş kuralı:** Bu alan, dosyanın fiziksel olarak nereden okunacağını belirler. `Remote` seçildiğinde sistem SFTP bağlantısı kurar; `Local` seçildiğinde disk I/O yapar. Yanlış seçim dosyanın bulunamamasına yol açar.

`FileType` (enum, zorunlu)

Değer	Teknik Anlam	İş Anlamı	Davranış Etkisi
`Card (1)`	Kart işlem dosyası	Bankadan gelen otorisyon/işlem kayıtları	`OceanTxnGuid` korelasyonu yapılır. Duplikat tespiti sırasında OceanTxnGuid kullanılır. Clearing eşleştirmesi sırasında "card" tarafı olarak aranır.
`Clearing (2)`	Takas/mutabakat dosyası	Kart ağından gelen takas verileri	`OceanTxnGuid` korelasyonu yapılır. Duplikat tespiti sırasında `ClrNo:ControlStat` birleşimi kullanılır. Mevcut kart satırlarıyla otomatik eşleştirilir.

- **Null/default gelirse:** Profil anahtarı oluşturulamaz, `CONFIGURATION` hatası döner.
- **İş kuralı:** Kart dosyası yüklendiğinde sistem mevcut clearing satırlarıyla eşleştirme yapar. Clearing dosyası yüklendiğinde tüm mevcut kart satırlarıyla eşleştirme yapar. **Sıralama kritiktir** — önce Card, sonra Clearing yüklenmesi ideal senaryodur; ancak ters sıra da çalışır çünkü her iki yönde de eşleştirme mantığı vardır.

`FileContentType` (enum, zorunlu)

Değer	Teknik Anlam	İş Anlamı
`Bkm (1)`	BKM formatı	Bankalararası Kart Merkezi dosya formatı
`Msc (2)`	Mastercard formatı	Mastercard ağı dosya formatı
`Visa (3)`	Visa formatı	Visa ağı dosya formatı

- **Davranış etkisi:** Parse kuralları bu enum'a göre belirlenir. Her ağın sabit genişlikli kayıt formatı farklıdır. Yanlış seçilirse parse başarısız olur, her satır `Failed` statüsünde kaydedilir. Ayrıca mutabakat değerlendirmesinde hangi evaluator'ın (BkmEvaluator, VisaEvaluator, MscEvaluator) devreye gireceği bu alana göre kararlaştırılır.

`FilePath` (string, opsiyonel)

- **Null/boş gelirse:** Konfigürasyondaki profil ayarlarına göre varsayılan dizin taranır ve dosya pattern'ine uyan tüm dosyalar işlenir.
- **Dolu gelirse:** Yalnızca belirtilen dosya işlenir.
- **İş kuralı:** Birden fazla dosya eşleşirse, paralel işleme konfigürasyonuna bağlı olarak dosyalar eş zamanlı veya sıralı işlenir. Tek dosya hedefleniyorsa `FilePath` belirtilmelidir.

1.3 Kod Akışı

1. Request null kontrolü → boş ise default `FileIngestionRequest` oluşturulur
2. Profil anahtarı oluşturma (`FileType:FileContentType`)
3. Profil ve parsing kuralları konfigürasyondan alınır
4. `IFileTransferClient` dosya kaynağına göre çözümlenir
5. Dosya(lar) keşfedilir (listing veya path resolve)
6. **Her dosya için:**
   - Header/Footer boundary kayıtları okunur
   - `FileKey` üretilir (header+footer hash) — dosya benzersizliğini garanti eder
   - Veritabanında aynı `FileKey + FileName + SourceType + FileType` ile kayıt var mı kontrol edilir
   - **Varsa ve kurtarma gerekiyorsa:** Eksik satırlar tamamlanır, hatalı satırlar yeniden denenir
   - **Varsa ve arşivleme gerekiyorsa:** Sadece arşiv akışı tetiklenir
   - **Varsa ve tam duplikasysa:** Mevcut kayıt güncellenerek dönülür
   - **Yoksa:** Yeni dosya entity'si oluşturulur, satır satır parse edilir
7. Her satır için:
   - Sabit genişlikli kayıt parse edilir
   - Korelasyon anahtarı atanır (`OceanTxnGuid` varsa direkt; yoksa `Rrn:CardNo:ProvisionCode:Arn:Mcc:Amount:Currency` birleşimi)
   - `ReconciliationStatus = Ready` atanır (korelasyon başarılıysa)
8. Batch halinde veritabanına yazılır (BulkInsert desteği var)
9. Dosya arşivlenir (hedef depoya kopyalama)
10. Duplikat tespiti çalıştırılır
11. Clearing eşleştirmesi yapılır
12. Dosya durumu finalize edilir

1.4 Veri Etkisi

- **Yazma:** `ingestion.file` tablosuna dosya kaydı, `ingestion.file_line` tablosuna satır kayıtları, `ingestion.card_*_detail` veya `ingestion.clearing_*_detail` tablolarına parse edilmiş detay kayıtları yazılır.
- **Güncelleme:** Mevcut dosya varsa sayaçlar güncellenir. Duplikat tespiti sırasında `duplicate_status` ve `duplicate_group_id` güncellenir. Clearing eşleştirmesi sırasında `matched_clearing_line_id` güncellenir.

1.5 Response Alan Analizi

Alan	Teknik Anlam	İş Anlamı	Yorumlama
`FileId`	Oluşturulan dosya kaydının GUID'i	Dosyanın sistem içindeki benzersiz tanımlayıcısı	Bu değerle evaluate/archive endpointlerine referans verilir
`FileKey`	Header+Footer hash'i	Dosya içeriğinin benzersiz parmak izi	Aynı `FileKey` gelirse dosya duplikat sayılır
`FileName`	Fiziksel dosya adı	Kaynak dosyanın adı	Operasyonel takip için kullanılır
`Status`	`FileStatus` enum	Dosyanın işlenme durumu	`Success` → her şey tamam; `Failed` → sayaç uyuşmazlığı, parse hatası veya arşiv hatası var; `Processing` → hâlâ işleniyor (bu response'ta görülmemeli)

`StatusName`	Status enum'ın string karşılığı	Okunabilir durum bilgisi	—
`Message`	Açıklama mesajı	İşlem özeti	Hata varsa nedenini açıklar
`TotalCount`	İşlenen detay satır sayısı	Dosyadaki toplam işlem adedi	`ExpectedCount` ile karşılaştırılmalı
`SuccessCount`	Başarılı parse edilen satır sayısı	Doğru okunan kayıt sayısı	`TotalCount - SuccessCount > 0` ise parse hataları var
`ErrorCount`	Hata sayısı	Parse veya persist hatası yaşayan satır sayısı	0'dan büyük değer. Yüksekse acil müdahale gerekir
`Errors`	Hata detay listesi	Hangi satırda (FileLineId), hangi adımda (Step) ne oldu	Root cause analizi için kullanılır

Bu response'a bakan kişi ne anlamalı:

- `Status = Success` ve `ErrorCount = 0` → Dosya sorunsuz alındı, tüm satırlar parse edildi, arşivlendi, mutabakat için hazır.
- `Status = Failed` ve `ErrorCount > 0` → Hatalı satırlar var. `Errors` listesinde satır numarası ve hata kodu incelenmeli.
- `Status = Failed` ve `Message` içinde "archive" geçiyorsa → Dosya parse edildi ama hedef depoya kopyalanamadı. Veri kaybolmaz ama arşiv bütünlüğü eksik kalır.
- `TotalCount < ExpectedCount (footer'daki değer)` → Dosya eksik okunmuş. İletişim hatası veya truncation olabilir.

1.6 Olası Senaryolar

Senaryo	Sonuç
Geçerli BKM kart dosyası, tüm satırlar temiz	`Status=Success`, `ErrorCount=0`, tüm satırlar `ReconciliationStatus=Ready`
Aynı dosya tekrar gönderilir	Duplikat tespit edilir, mevcut kayıt döner, yeniden yazılmaz
Parse hatası olan satırlar	Hatalı satırlar `Failed`, dosya `Failed` (hata sayısı > 0), hatalı satırlar sonraki gönderimde otomatik retry edilir
SFTP bağlantı hatası	`FILE_RESOLUTION` step'inde hata, boş response döner
Dosya bulunamaz	`FILE_NOT_FOUND` kodu ile hata
Clearing dosyası yüklenir, kart dosyası zaten var	Korelasyon anahtarı üzerinden otomatik eşleştirme yapılır
OceanTxnGuid boş olan satır	Fallback korelasyon anahtarı kullanılır; o da eksikse `ReconciliationStatus=Failed`
Arşiv kopyalama başarısız (3 deneme sonra)	Dosya `IsArchived=false`, `Status=Failed`, mesajda arşiv hatası belirtilir
Arşiv kopyalama başarısız (3	

Arşiv kopyalama başarısız (3 deneme sonrası)	Dosya `IsArchived=false`, `Status=Failed`, mesajda arşiv hatası belirtilir

2. Mutabakat Değerlendirme (Evaluate) Endpoint Analizi

2.1 Endpoint Amacı

`POST /v1/Reconciliation/Evaluate`

Sisteme alınmış dosya satırlarını analiz ederek her satır için **hangi operasyonların yapılması gerektiğine karar verir**. Bu endpoint işlemleri **yürütmez** — yalnızca planlar. Kart ve clearing verilerini karşılaştırarak tutar uyuşmazlığı, para birimi farkı gibi durumları saptar; her sorun için bir operasyon planı oluşturur; gerekiyorsa manuel inceleme adımı ekler.

2.2 Request Alan Analizi

`IngestionFileIds` (Guid[], opsiyonel)

- **Boş dizi veya null gelirse:** Sistem **tüm veritabanındaki** `ReconciliationStatus = Ready` olan satırların dosya ID'lerini otomatik bulur ve hepsini değerlendirir. Bu, "bekleyen her şeyi değerlendir" anlamına gelir.
- **Dolu gelirse:** Yalnızca belirtilen dosyalara ait hazır satırlar değerlendirilir.
- **Guid.Empty içerirse:** Empty GUID'ler filtrelenir, geriye kalan geçerli ID'ler kullanılır.
- **İş kuralı:** Hedef dosya belirtilmezse sistem kendi scope'unu genişletir. Bu, cron job senaryosunda tüm birikmiş işlerin toplu değerlendirilmesi için kullanılır. Spesifik dosya belirtmek, bir dosyanın hızlıca değerlendirilmesini sağlar.

`Options` (EvaluateOptions, opsiyonel)

`ChunkSize` (int?, varsayılan: 50.000)

- **Teknik anlam:** Bir seferde veritabanından claim edilen satır sayısı.
- **İş anlamı:** Bellek ve veritabanı kilit yönetimi. Çok büyük değer bellek taşmasına yol açabilir; çok küçük değer performansı düşürür.
- **Sınır:** Minimum 100, maksimum 10.000 (Clamp uygulanır). 0 veya negatif gelirse exception fırlatılır.

`ClaimTimeoutSeconds` (int?, varsayılan: 1.800 = 30 dakika)

- **Teknik anlam:** Bir satırın `Processing` statüsünde claim edilmiş olarak kalabileceği maksimum süre. Bu süre aşılırsa satır "stale" kabul edilir ve tekrar claim edilebilir.
- **İş anlamı:** Worker çökerse veya yanıt vermezse, bu timeout sonrası başka bir worker satırı devralabilir. Çok kısa tutulursa aynı satır birden fazla worker tarafından işlenebilir.

`ClaimRetryCount` (int?, varsayılan: 5)

- **Teknik anlam:** `DbUpdateException` veya `InvalidOperationException` alındığında claim işleminin kaç kez deneneceği.
- **İş anlamı:** Eş zamanlı çalışan worker'lar arasında veritabanı çakışması olursa otomatik retry yapılır.

`OperationMaxRetries` (int?, varsayılan: 5)

- **Teknik anlam:** Oluşturulan operasyonların kaç kez yeniden denenebileceği.
- **İş anlamı:** Execute aşamasında bir operasyon başarısız olursa, bu değer kadar retry hakkı olur. Tükenirse

operasyon `Failed` olur.

2.3 Kod Akışı (Detaylı)

1. **Hedef dosya çözümleme:** `IngestionFileIds` boşsa tüm `Ready` satırların dosya ID'leri bulunur.
2. **Her dosya için chunk döngüsü:**
   - `SERIALIZABLE` izolasyon seviyesinde transaction açılır
   - `Ready` veya stale `Processing` (UpdateDate < şimdi - timeout) satırlar seçilir
   - Seçilen satırların `ReconciliationStatus` → `Processing`, `Message` → claim marker (unique GUID) olarak güncellenir
   - Claim marker ile doğrulama yapılır (başka worker'ın aynı anda claim etmediğinden emin olunur)
3. **Context oluşturma:** Her satır için evaluation context build edilir (kart detayı, clearing detayı, eşleşme bilgisi)
4. **Değerlendirme:** ContentType'a göre doğru evaluator seçilir (BkmEvaluator, VisaEvaluator, MscEvaluator)
5. **Sonuç persistansı:**
   - `ReconciliationEvaluation` kaydı oluşturulur
   - Her operasyon için `ReconciliationOperation` kaydı oluşturulur
   - Manuel operasyonlar için `ReconciliationReview` kaydı oluşturulur (Pending durumda)
   - İlk operasyon `Planned`, sonrakiler `Blocked` statüsünde başlar
   - Branch mantığı: Manuel gate'in approve/reject dalları ayrı operasyonlar olarak planlanır
6. **Satır güncelleme:** Başarılı satırlar `ReconciliationStatus = Success`, hatalı satırlar `Failed` olarak güncellenir. Hatalı satırlar için `ReconciliationAlert` oluşturulur.

2.4 Veri Etkisi

- **Güncelleme:** `ingestion.file_line.reconciliation_status` güncellenir (Ready → Processing → Success/Failed)
- **Yazma:** `reconciliation.evaluation`, `reconciliation.operation`, `reconciliation.review`, `reconciliation.alert` tablolarına kayıt oluşturulur

2.5 Response Alan Analizi

Alan	Teknik Anlam	İş Anlamı	Yorumlama
`EvaluationRunId`	Bu çalıştırmanın benzersiz ID'si	Aynı evaluate çağrısında oluşturulan tüm kayıtların grup ID'si	Execute'da bu ID ile filtreleme yapılabilir
`CreatedOperationsCount`	Oluşturulan operasyon sayısı	Kaç adet aksiyon planlandı	0 ise ya hiç satır yoktu ya da tümü zaten değerlendirilmişti
`SkippedCount`	Atlanan (hatalı) satır sayısı	Değerlendirme sırasında exception alan satırlar	Bu sayı yüksekse veri kalitesi sorunu var
`Message`	İşlem özeti	Başarı/hata mesajı	`ErrorCount > 0` ise mesaj "hatalarla tamamlandı" formatında olur
`ErrorCount`	Hata sayısı	Kaç satırda/adımda hata oluştu	0 olması gereken değer. Yüksekse acil müdahale gerekir
`Errors`	Detaylı hata listesi	Hangi satırda (FileLineId), hangi adımda (Step) ne oldu	Root cause analizi için kullanılır

CreatedOperationsCount yüksekse: Çok sayıda uyumsuzluk tespit edilmiş demektir. Bu, dosya kalitesinin düşük olduğunun veya clearing dosyasının henüz yüklenmediğinin göstergesidir.

CreatedOperationsCount = 0 ve SkippedCount = 0: Tüm satırlar zaten değerlendirilmiş veya hiç hazır satır yok.

SkippedCount yüksekse: Context build aşamasında hatalar var. Bu genellikle veri bütünlüğü sorunlarına (eksik dosya kaydı, bozuk parsed data) işaret eder.

2.6 Olası Senaryolar

Senaryo	Sonuç
Clearing dosyası henüz yüklenmemiş, sadece kart dosyası var	`UNMATCHED_CARD` operasyonları oluşturulur
Tutar uyuşmazlığı (> 0.01 fark)	`AmountMismatch` operasyonu planlanır
Eşleşmiş ve temiz kayıt	Operasyon oluşturulmayabilir veya sadece bilgilendirme operasyonu planlanır
Manuel inceleme gerektiren durum	`CreateManualReview` operasyonu + Review kaydı oluşturulur
Aynı satır ikinci kez evaluate edilmeye çalışılır	Zaten `Success` durumunda, claim sırasında atlanır
Worker çökmesi sonrası	Stale claim'ler timeout sonrası tekrar claim edilir ve değerlendirilir

---

3. Mutabakat Yürütme (Execute) Endpoint Analizi

3.1 Endpoint Amacı

`POST /v1/Reconciliation/Operations/Execute`

Evaluate aşamasında planlanan operasyonları **fiilen yürütür**. Her operasyon sırasıyla (sequence) çalıştırılır. Manuel onay bekleyen operasyonlar bloke olur. Başarılı operasyonlar sonraki adımı tetikler. Başarısız operasyonlar retry mekanizmasına girer.

Execute tamamlandıktan sonra, konfigürasyonda `AutoArchiveAfterExecute = true` ise ve en az bir operasyon başarılı olduysa, **arka planda otomatik arşiv** tetiklenir (fire-and-forget).

3.2 Request Alan Analizi

`GroupIds` (Guid[], opsiyonel)

- **İş anlamı:** Evaluate çalıştırmasının ürettiği grup ID'leri. Belirli bir evaluate batch'inin tüm operasyonlarını hedefler.
- **Boş gelirse ve diğer filtreler de boşsa:** Tüm çalıştırılabilir operasyonlar yürütülür.
- **Öncelik:** `OperationIds > EvaluationIds > GroupIds > All` sıralaması uygulanır.

`EvaluationIds` (Guid[], opsiyonel)

- **İş anlamı:** Belirli evaluation kayıtlarının operasyonlarını hedefler. Tek bir dosya satırı için oluşturulmuş tüm operasyonları çalıştırır.
- **Boş ve GroupIds de boş ise:** OperationIds kontrolüne düşer.

`OperationIds` (Guid[], opsiyonel)

- **İş anlamı:** En spesifik filtreleme. Belirli operasyonları direkt hedefler.
- **Kullanım:** Hata sonrası tek bir operasyonu tekrar çalıştırmak için.
- **Boş gelirse:** Bir üst seviye filtre kontrol edilir.

Tüm filtreler boş gelirse: Sistem tüm çalıştırılabilir (Planned/Blocked/Executing + lease expired + next_attempt_at geçmiş) operasyonları yürütür. Bu "her şeyi çalıştır" modudur ve genellikle cron job tarafından tetiklenir.

`Options` (ExecuteOptions, opsiyonel)

`MaxEvaluations` (int?, varsayılan: 500.000)

- **Teknik anlam:** Bu çalıştırmada işlenecek maksimum evaluation sayısı.
- **İş anlamı:** Performans sınırı. Çok büyük tutulursa long-running request olur.

`LeaseSeconds` (int?, varsayılan: 900 = 15 dakika)

- **Teknik anlam:** Bir operasyonun claim edildikten sonra lease süresi. Bu süre içinde tamamlanmazsa başka worker devralabilir.
- **İş anlamı:** Worker çökmelerinde kilitlenmeyi önler.

3.3 Kod Akışı (Detaylı)

1. **Selection modu belirleme:** Request ID'lerine göre `OperationIds` → `EvaluationIds` → `GroupIds` → `All` sıralamasıyla mod seçilir.
2. **Hedef evaluation'lar çözümleme:** Aktif (Planned/Blocked/Executing) operasyonlar sorgulanır. Lease ve NextAttemptAt süreleri kontrol edilir.
3. **Her evaluation için operasyon döngüsü:**
   a. Operasyonlar sıralı yüklenir (`SequenceIndex` sırasıyla)
   b. Sonraki çalıştırılabilir operasyon bulunur:
   - `Completed` / `Cancelled` / `Failed` → atlanır
   - `Executing` → lease süresi dolmuşsa devralınır, dolmamışsa bekle
   - `Blocked` → parent tamamlandıysa promote edilir, manuel ise onaylanmışsa devralınır
   - `Planned` → çalıştırılmaya hazır
   - `NextAttemptAt > şimdi` → henüz zamanı gelmemiş, dur
     c. **Claim:** Atomik güncelleme ile operasyon `Executing` statüsüne alınır, `LeaseOwner` ve `LeaseExpiresAt` atanır.
     d. **Yürütme:**
   - **Manuel gate operasyonu (`CreateManualReview`):**
     - Review kaydı sorgulanır
     - `Pending` ve süre dolmamış → operasyon `Blocked` 'a döner, `WAITING_MANUAL_DECISION` result
     - `Pending` ve süre dolmuş → `ExpirationAction` 'a göre otomatik karar (Approve/Reject/Cancel)
     - `Approved` → gate tamamlanır, approve dalı `Blocked` 'a (çalıştırılmaya hazır), reject dalı `Cancelled` 'a geçer
     - `Rejected` → gate tamamlanır, reject dalı `Blocked` 'a, approve dalı `Cancelled` 'a geçer
     - `Cancelled` → gate ve tüm dallar `Cancelled` . `ExpirationFlowAction = CancelRemaining` ise kalan tüm operasyonlar da iptal
   - **Normal operasyon:**
     - İdempotency kontrolü: Aynı `IdempotencyKey` ile `Completed` operasyon varsa → `Skipped`
     - `OperationExecutor.ExecuteAsync()` çağrılır (dış sistem entegrasyonu)
     - Başarılı → `Completed` , hata temizlenir

- Başarısız → retry planlanır (exponential backoff: `30 * 2^retryCount` saniye)
- Retry limiti (MaxRetries) aşılırsa → `Failed` + alert oluşturulur

e. **Execution kaydı:** Her deneme `reconciliation.operation_execution` tablosuna yazılır

4. **Alert servisi çalıştırılır:** Tüm operasyonlar bittikten sonra alert servisi tetiklenir (e-posta bildirimleri vb.)
5. **Otomatik arşiv:** `AutoArchiveAfterExecute = true` ve `TotalSucceeded > 0` ise arka planda `RunArchiveCommand` tetiklenir (fire-and-forget)

3.4 Veri Etkisi

- **Güncelleme:** `reconciliation.operation` statü, lease, retry alanları güncellenir. `reconciliation.review` kararı güncellenir.
- **Yazma:** `reconciliation.operation_execution` tablosuna her deneme kaydedilir. Başarısız operasyonlar için `reconciliation.alert` oluşturulur.

3.5 Response Alan Analizi

Alan	Teknik Anlam	İş Anlamı	Yorumlama
`TotalAttempted`	Denenen toplam operasyon sayısı	Kaç operasyon yürütülmeye çalışıldı	
`TotalSucceeded`	Başarılı + atlanmış (Completed/Skipped) sayısı	Kaç operasyon hedefe ulaştı	`Skipped` da başarılı sayılır (idempotent)
`TotalFailed`	Başarısız operasyon sayısı	Kaç operasyon hata aldı	0'dan büyükse `Results` listesinde detay var
`Results`	Başarısız operasyonların listesi	Sadece hata alanlar burada	Başarılılar response'ta yer almaz (boyut optimizasyonu)

TotalSucceeded = TotalAttempted: Tüm operasyonlar başarılı. İdeal durum.
TotalFailed > 0: Hatalı operasyonlar var. `Results` listesinde `OperationId` ve `Message` ile root cause araştırılmalı.
TotalAttempted = 0: Çalıştırılabilir operasyon bulunamadı. Ya hepsi tamamlanmış, ya hepsi bloke, ya da timing sorunu (NextAttemptAt henüz gelmemiş).

3.6 Olası Senaryolar

Senaryo	Sonuç
Tüm operasyonlar otomatik ve başarılı	`TotalSucceeded = TotalAttempted`, `TotalFailed = 0`
Manuel onay bekleyen operasyon	Operasyon `Blocked` döner, sonraki operasyonlar yürütülmez
Manuel onay süresi dolmuş, ExpirationAction = Approve	Otomatik onaylanmış, approve dalı aktifleşmiş
Dış sistem hatası, retry limiti aşılmamış	`Failed` ama `Planned` statüsüne döner, exponential backoff ile yeniden denenir
Dış sistem hatası, retry limiti aşılmış	Operasyon `Failed`, alert oluşturulur
Aynı operasyon tekrar yürütülür (idempotency)	`Skipped` olarak kaydedilir, tekrar yürütülmez
Worker çökmesi	Lease süresi dolunca başka worker devralır

---

4. Manuel Onay (Approve) Endpoint Analizi

4.1 Endpoint Amacı

`POST /v1/Reconciliation/Reviews/Approve`

Mutabakat sürecinde manuel inceleme gerektiren bir operasyonu **onaylar**. Onay, ilgili operasyonun approve dalındaki operasyonların çalıştırılmasına izin verir.

4.2 Request Alan Analizi

`OperationId` (Guid, zorunlu)

- **Teknik anlam:** Manuel inceleme bekleyen operasyonun ID'si.
- **Boş (Guid.Empty) gelirse:** Review bulunamaz, `REVIEW_NOT_FOUND` hatası döner.
- **Yanlış ID gelirse:** Review bulunamaz → `NotFound` result; veya review zaten karara bağlanmış → `Invalid` result.
- **İş kuralı:** Bu ID, Pending Reviews listesinden alınmalıdır. Rastgele bir operasyon ID'si göndermek operasyonun review'ının bulunamama durumuna yol açar.

`ReviewerId` (Guid?, opsiyonel)

- **Null/empty gelirse:** Audit context'teki kullanıcı ID'si otomatik atanır.
- **Dolu gelirse:** Belirtilen reviewer ID kaydedilir.
- **İş anlamı:** Kimin onayladığının audit trail için saklanması.

`Comment` (string, opsiyonel)

- **Null/boş gelirse:** Yorum kaydedilmez, işlem devam eder.
- **İş anlamı:** Onay gerekçesi. Audit ve raporlama için önemlidir.

4.3 Kod Akışı

1. Request null kontrolü
2. **Transaction başlatılır** (execution strategy ile)
3. `ReconciliationReview` tablosunda `OperationId` + `Decision = Pending` olan kayıt aranır
4. Bulunamazsa:
   - Review hiç yoksa → `NotFound`
   - Review var ama Pending değilse → `Invalid` (zaten karara bağlanmış)
5. Review güncellenir: `Decision = Approved`, `ReviewerId`, `Comment`, `DecisionAt`
6. İlgili operasyonun `NextAttemptAt` = şimdi olarak ayarlanır (hemen yürütülmesini tetikler), lease bilgileri temizlenir
7. Operasyon güncellenemezse (zaten Completed/Failed/Cancelled):
   - `Invalid` result: "operasyon yeniden kuyruğa alınamaz"
8. Transaction commit edilir

4.4 Response Alan Analizi

Alan	Teknik Anlam	İş Anlamı	Yorumlama
`OperationId`	İşlenen operasyon ID'si	Hangi operasyon onaylandı	
`Result`	İşlem sonucu	`Approved` / `NotFound` / `Invalid` / `Failed`	

`Message`	Açıklama	İşlem detayı	

Result = "Approved": Onay başarılı. Sonraki Execute çalıştırmasında approve dalı yürütülecek.
Result = "NotFound": OperationId yanlış veya review hiç oluşturulmamış.
Result = "Invalid": Review zaten karara bağlanmış veya operasyon terminal durumda.
Result = "Failed": Teknik hata (veritabanı, transaction vb.)

4.5 Olası Senaryolar

Senaryo	Sonuç
Geçerli pending review + approve	`Result = "Approved"`, operasyon NextAttemptAt = now
Zaten onaylanmış review tekrar approve	`Result = "Invalid"`
Var olmayan operasyon ID	`Result = "NotFound"`
Operasyon zaten `Completed` durumda	`Result = "Invalid"` (yeniden kuyruğa alınamaz)
Veritabanı hatası	`Result = "Failed"` + Errors listesinde detay

---

5. Manuel Red (Reject) Endpoint Analizi

5.1 Endpoint Amacı

`POST /v1/Reconciliation/Reviews/Reject`

Manuel inceleme gerektiren bir operasyonu **reddeder**. Red, reject dalındaki operasyonların çalıştırılmasını tetikler; approve dalındaki operasyonlar iptal edilir.

5.2 Request Alan Analizi

Approve ile aynı yapıdadır:

`OperationId`, `ReviewerId`, `Comment`

Davranışlar Approve ile birebir aynıdır, tek fark: `Decision = Rejected` olarak set edilir.

5.3 Kod Akışı

Approve ile aynı akış çalışır. Fark:

- `ReviewDecision.Rejected` kaydedilir.
- Execute aşamasında reject dalı aktifleşir, approve dalı `Cancelled` olur.

5.4 Response

Approve ile aynı format. `Result` alanı `"Rejected"`, `"NotFound"`, `"Invalid"` veya `"Failed"` olabilir.

---

6. Bekleyen İncelemeler (Pending Reviews) Endpoint Analizi

6.1 Endpoint Amacı

`GET /v1/Reconciliation/Reviews/Pending`

Manuel onay/red bekleyen tüm operasyonları listeler. Bu, **operatörün iş kuyruğudur** — buradaki her kayıt bir insan kararı beklemektedir.

6.2 Request Alan Analizi (Query String)

`Date` (DateOnly?, opsiyonel)

- **Null gelirse:** Tüm pending review'lar döner.
- **Dolu gelirse:** Sadece o güne ait review'lar filtrelenir.
- **İş kuralı:** Günlük operasyonel takip için kullanılır.

Sayfalama: `Page`, `Size`, `SortBy`, `OrderBy`

- `Page`: Minimum 1, default 1.
- `Size`: Minimum 1, maksimum 1000 (Clamp uygulanır).
- Sıralama review `CreateDate`'e göredir.

6.3 Response Alan Analizi

`Data` alanı `PaginatedList<ManualReview>` tipindedir.

ManualReview alanları:

Alan	Teknik Anlam	İş Anlamı	Yorumlama
`OperationId`	Manuel operasyonun ID'si	Approve/Reject çağrısında kullanılacak ID	
`FileLineId`	İlgili dosya satırının ID'si	Hangi işlem satırı hakkında karar bekleniyor	
`OperationCode`	Operasyon kodu	Ne tür bir operasyon	`CreateManualReview` ise bu bir gate operasyonu
`OperationPayload`	Operasyon payload'u	Gate ise approve/reject dallarının operation kodlarını içerir	JSON formatında okunabilir
`CreatedAt`	Oluşturulma zamanı	Review ne zaman oluştu	Yaşlanma takibi için
`ApproveBranchOperations`	Onay dalı operasyonları	Onaylanırsa ne çalışacak	Operatörün kararını bilgilendirmek için
`RejectBranchOperations`	Red dalı operasyonları	Reddedilirse ne çalışacak	
`ExpiresAt`	Son karar tarihi	Bu tarihe kadar karar verilmezse otomatik aksiyon	null ise süre sınırı yok
`ExpirationAction`	Süre dolunca yapılacak	`Cancel` / `Approve` / `Reject`	Otomatik karar yönünü belirler
`ExpirationFlowAction`	Süre dolunca akış davranışı	`Continue` / `CancelRemaining`	`CancelRemaining` → tüm kalan operasyonlar iptal

`ApprovalMessage`	Onay mesajı	UI'da gösterilecek bilgilendirme	
`RejectionMessage`	Red mesajı	UI'da gösterilecek bilgilendirme	

Bu listeye bakan operatör ne anlamalı:

- Her satır bir insan kararı bekliyor.
- `ExpiresAt` yaklaşmışsa acil karar gerekli.
- `ApproveBranchOperations` ve `RejectBranchOperations` karar sonuçlarını gösterir — operatör bunları okuyarak hangi yönde karar vereceğini bilgilendirebilir.
- `ExpirationAction = Approve` ve süre dolmak üzereyse → zaten otomatik onaylanacak, müdahale gerekmeyebilir.
- `ExpirationAction = Cancel` ve süre dolmak üzereyse → iptal olacak, önce karar verilmeli.

---

7. Uyarılar (Alerts) Endpoint Analizi

7.1 Endpoint Amacı

`GET /v1/Reconciliation/Alerts`

Mutabakat sürecinde oluşan uyarıları listeler. Uyarılar, başarısız değerlendirmeler ve başarısız operasyon yürütmeleri sonucunda otomatik oluşturulur.

7.2 Request Alan Analizi

`Date` (DateOnly?, opsiyonel)

- **İş anlamı:** Belirli bir güne ait uyarıları filtreler.

`AlertStatus` (enum?, opsiyonel)

Değer	Anlam
`Pending (0)`	Henüz işlenmemiş
`Processing (1)`	İşleniyor
`Consumed (2)`	Başarıyla tüketilmiş (bildirim gönderilmiş)
`Failed (3)`	İşleme başarısız
`Ignored (4)`	Kasıtlı olarak yok sayılmış

7.3 Response Alan Analizi

Alan	Teknik Anlam	İş Anlamı	Yorumlama
`Severity`	Önem derecesi	`Error` ise acil müdahale	
`AlertType`	Uyarı türü	`EvaluationFailed` veya `OperationExecutionFailed`	Sorunun evaluate mı execute mı aşamasında olduğunu belirtir
`Message`	Hata mesajı	Kök neden bilgisi	
`OperationId`	İlgili operasyon	`Guid.Empty` ise evaluate aşamasında olusmus	

`EvaluationId`	İlgili değerlendirme	Hangi evaluation kaydıyla ilişkili	

AlertType = "EvaluationFailed": Satır değerlendirilemedi. Veri kalitesi sorunu veya context build hatası. `OperationId` boş çünkü operasyon oluşturulamadan hata alınmıştır.

AlertType = "OperationExecutionFailed": Operasyon retry limitini aştı. Dış sistem hatası veya kalıcı bir sorun var. Manuel müdahale gerekir.

---

8. Arşiv Önizleme (Archive Preview) Endpoint Analizi

8.1 Endpoint Amacı

`POST /v1/Archive/Preview`

Arşivlenmeye aday dosyaları **silmeden önce** listeler. Her aday için uygunluk kontrolü yapar ve silinecek kayıt sayılarını gösterir. **Hiçbir veri değiştirmez.**

8.2 Request Alan Analizi

`IngestionFileIds` (Guid[]?, opsiyonel)

- **Null/boş gelirse:** Sistem `BeforeDate` stratejisine göre tüm aday dosyaları bulur.
- **Dolu gelirse:** Yalnızca belirtilen dosyalar kontrol edilir.

`BeforeDate` (DateTime?, opsiyonel)

- **Null gelirse:** Konfigürasyondaki strateji uygulanır:
  - `RetentionDays` stratejisi → `Şimdi - RetentionDays gün` (varsayılan 90 gün)
  - `None` stratejisi → tarih filtresi yok
- **Dolu gelirse (ve `UseConfiguredBeforeDateOnly = false`):** Belirtilen tarih kullanılır.
- **`UseConfiguredBeforeDateOnly = true` ise:** Request'teki tarih **yok sayılır**, her zaman konfigürasyon kullanılır.

`Limit` (int?, opsiyonel)

- **Null/0 gelirse:** Varsayılan `PreviewLimit` (5.000) kullanılır.
- **Dolu gelirse:** 1-1000 arasında clamp edilir.

8.3 Response Alan Analizi

`Candidates` listesi:

Alan	Teknik Anlam	İş Anlamı
`IngestionFileId`	Dosya ID'si	Hangi dosya aday
`IsEligible`	Arşivlenebilir mi	`true` → güvenle silinebilir; `false` → henüz silinemez
`FailureReasons`	Neden silinemez	Hangi koşullar sağlanmadı (terminal status kontrolü, min age kontrolü vb.)
`Counts`	Kayıt sayıları	Silinecek dosya satırı, detay, evaluation, operation, review, execution, alert sayıları

IsEligible = false görürsem: Dosya henüz arşivleme koşullarını sağlamıyor. `FailureReasons` okunmalı:

- Status terminal değilse (hâlâ Processing veya Ready) → operasyonlar tamamlanmadan silinmemeli.
- MinLastUpdateAgeHours (72 saat) sağlanmamışsa → dosya çok taze, beklemeli.

---

9. Arşiv Çalıştır (Archive Run) Endpoint Analizi

9.1 Endpoint Amacı

`POST /v1/Archive/Run`

Uygun dosyaları ve ilişkili tüm kayıtları **kalıcı olarak siler**. Bu geri dönüşü olmayan bir işlemdir.

9.2 Request Alan Analizi

`IngestionFileIds`, `BeforeDate`

Preview ile aynı mantık.

`MaxFiles` (int?, opsiyonel)

- **Null/0:** Varsayılan `MaxRunCount` (50.000) kullanılır.
- **İş anlamı:** Tek seferde silinecek maksimum dosya sayısını sınırlar.

`ContinueOnError` (bool?, opsiyonel)

- **Null:** Konfigürasyon değeri kullanılır (varsayılan `false`).
- **false:** İlk hata anında durur. Güvenli mod.
- **true:** Hatalı dosyayı atlayıp devam eder. Agresif temizlik modu.
- **İş kuralı:** Üretimde `false` önerilir. Toplu temizlikte bir dosyadaki hata diğerlerini bloke etmemesi için `true` kullanılabilir.

9.3 Kod Akışı

1. Audit context kontrol edilir (kim çalıştırıyor?)
2. Aday dosya ID'leri çözümlenir
3. **Her aday için:**
   a. Uygunluk kontrolü yapılır
   b. Uygunsa: `ArchiveExecutor.ExecuteAsync()` ile tüm ilişkili kayıtlar silinir
   c. Hata alınırsa: `MaxRetryPerFile + 1` deneme yapılır, denemeler arası `RetryDelaySeconds` beklenir
   d. Sonuç kayıt: `archive.archive_log` tablosuna işlem logu yazılır
   e. `ContinueOnError = false` ve hata varsa → döngü durur

9.4 Response Alan Analizi

Alan	Teknik Anlam	İş Anlamı
`ProcessedCount`	İşlenen dosya sayısı	Kaç dosya denendi
`ArchivedCount`	Silinen dosya sayısı	Kaç dosya başarıyla temizlendi
`SkippedCount`	Atlanan dosya sayısı	Uygun olmayan dosyalar

`FailedCount`	Başarısız dosya sayısı	Silinemeyen dosyalar
`Items`	Dosya bazlı sonuç listesi	Her dosya için Status + FailureReasons

ArchivedCount = ProcessedCount: Tüm dosyalar başarıyla silindi.
FailedCount > 0: Bazı dosyalar silinemedi. `Items` listesinde hata detayı var.
SkippedCount yüksekse: Çok sayıda dosya henüz arşivleme koşullarını sağlamıyor.

---

10. Raporlama Endpoint Analizleri

10.1 İşlemler Listesi (GET /v1/Reporting/Reconciliation/Transactions)

Endpoint Amacı

Kart-clearing eşleştirilmiş tüm işlemleri filtreli ve sayfalı şekilde listeler. `vw_reconciliation_matched_pair` view'ından okur.

Request Alanları

Alan	Tip	Anlam	Null Davranışı
`DateFrom`	int?	İşlem tarihinden itibaren (YYYYMMDD formatı)	Filtre uygulanmaz
`DateTo`	int?	İşlem tarihine kadar	Filtre uygulanmaz
`Network`	enum?	BKM/VISA/MSC	Filtre uygulanmaz, tüm ağlar gelir
`MatchStatus`	enum?	MATCHED/UNMATCHED_CARD	Filtre uygulanmaz
`HasAmountMismatch`	bool?	Tutar uyuşmazlığı var mı	Filtre uygulanmaz
`HasCurrencyMismatch`	bool?	Para birimi uyuşmazlığı	Filtre uygulanmaz
`HasDateMismatch`	bool?	Tarih uyuşmazlığı	Filtre uygulanmaz
`HasStatusMismatch`	bool?	Durum uyuşmazlığı	Filtre uygulanmaz
`DuplicateStatus`	enum?	Unique/Primary/Secondary/Conflict	Filtre uygulanmaz
`SortBy`	string?	Sıralama alanı	Varsayılan: CardTransactionDate desc
`OrderBy`	enum?	Asc/Desc	Varsayılan: Desc
`Page` , `Size`	int	Sayfalama	Page min 1, Size min 1 max 1000

Filtre kombinasyonları: Tüm filtreler AND mantığıyla birleşir. `HasAmountMismatch=true&HasCurrencyMismatch=true` → hem tutar hem para birimi uyuşmazlığı olan kayıtlar.

Response: ReconciliationTransactionDto Alanları

Alan	İş Anlamı	Bu değeri görürsem ne anlamalıyım
`CardOceanTxnGuid`	Kart tarafının işlem ID'si	Benzersiz işlem tanımlayıcı
	Retrieval Reference	

`CardRrn`	Retrieval Reference Number	İşlem takip numarası
`CardArn`	Acquirer Reference Number	Otorisyon referans numarası
`CardOriginalAmount`	Kart tarafı orijinal tutar	İşlemin talep edilen tutarı
`ClearingSourceAmount`	Clearing tarafı kaynak tutar	Takas dosyasındaki tutar
`AmountDifference`	`CardOriginalAmount - ClearingSourceAmount`	Pozitifse kart fazla, negatifse clearing fazla
`HasAmountMismatch`	Tutar farkı > 0.01	`true` → tutar uyuşmazlığı var, araştırılmalı
`HasCurrencyMismatch`	Para birimi farklı	`true` → kart ve clearing farklı para birimi raporluyor
`HasDateMismatch`	Tarih farklı	`true` → işlem tarihleri uyuşmuyor
`HasStatusMismatch`	Durum çelişkisi	`true` → kart "başarılı" diyor ama clearing "normal değil" diyor (veya tersi)
`MatchStatus`	Eşleşme durumu	`MATCHED` → clearing bulundu; `UNMATCHED_CARD` → clearing bulunamadı
`DuplicateStatus`	Duplikat durumu	`Unique` → normal; `Primary` → duplikat grubunun ana kaydı; `Secondary` → duplikat kopya; `Conflict` → çakışma
`ReconciliationStatus`	Mutabakat durumu	`Success` → değerlendirildi; `Ready` → bekliyor; `Failed` → değerlendirme başarısız
`ClearingControlStat`	Clearing kontrol durumu	`Normal` → sorunsuz; farklı değer → dispute, chargeback vb.

10.2 Problemli Kayıtlar (GET /v1/Reporting/Reconciliation/Problems)

Endpoint Amacı

`vw_reconciliation_problem_records` view'ından okur. Şu koşullardan **herhangi birini** sağlayan kayıtları listeler:

- `UNMATCHED_CARD`
- `has_amount_mismatch = TRUE`
- `has_currency_mismatch = TRUE`
- `has_date_mismatch = TRUE`
- `has_status_mismatch = TRUE`
- `duplicate_status != 'Unique'`

Neden ayrı endpoint: Bu, "sadece sorunlu olanları göster" kısayoludur. Transactions endpoint'inde tüm filtreleri tek tek set etmek yerine, tek çağrıyla tüm problemleri gösterir.

Bu listeye bakan analist: Her kayıt bir sorun içeriyor. `HasAmountMismatch` en kritik olandır — tutar farkı finansal kayba işaret eder. `UNMATCHED_CARD` clearing dosyasının gecikmesine bağlı olabilir (geçici sorun) veya kayıp işleme (kalıcı sorun).

10.3 Eşleşmemiş Kayıtlar (GET /v1/Reporting/Reconciliation/Unmatched)

Endpoint Amacı

`vw_reconciliation_unmatched_card` view'ından okur. Yalnızca `match_status = 'UNMATCHED_CARD'` olan kayıtları listeler.

Neden ayrı endpoint: Eşleşmemiş kayıtlar özel takip gerektirir. Clearing dosyası gelmediğinde veya geciktiğinde bu liste büyür. Clearing dosyası yüklendikten sonra bu listenin küçülmesi beklenir. **Küçülmezse clearing dosyasında kayıp işlem var demektir.**

Analist aksiyonu:

- Liste büyüyorsa → clearing dosyaları gecikiyor mu kontrol et
- Clearing gelmiş ama liste küçülmemişse → korelasyon anahtarı eşleşmesi sorunlu
- Çok eski kayıtlar varsa → bu işlemler muhtemelen kayıp, manuel araştırma gerekir

10.4 Günlük Özet (GET /v1/Reporting/Reconciliation/Summary/Daily)

Endpoint Amacı

Her işlem tarihi için mutabakat KPI'larını üretir.

Response Alanları

Alan	İş Anlamı	Bu değeri görürsem ne anlamalıyım
`TransactionDate`	İşlem tarihi (YYYYMMDD)	Hangi günün özeti
`TotalCount`	Toplam kayıt	O günün toplam işlem hacmi
`MatchedCount`	Eşleşen	Clearing ile eşleşmiş kayıt sayısı
`UnmatchedCount`	Eşleşmeyen	Clearing bulunamayan kayıt sayısı
`AmountMismatchCount`	Tutar uyuşmazlığı	Tutar farkı > 0.01 olan kayıt sayısı
`CurrencyMismatchCount`	Para birimi uyuşmazlığı	Farklı para birimi raporlayan kayıt sayısı
`DateMismatchCount`	Tarih uyuşmazlığı	Tarih tutarsızlığı olan kayıt sayısı
`StatusMismatchCount`	Durum uyuşmazlığı	Durum çelişkisi olan kayıt sayısı
`ProblemCount`	Toplam problemli	Herhangi bir sorunu olan kayıt sayısı (union)
`CleanCount`	Temiz kayıt	Eşleşmiş ve hiçbir uyuşmazlığı olmayan kayıt sayısı

KPI yorumlama:

- `CleanCount / TotalCount` → Mutabakat başarı oranı. %99'un altı alarm gerektirir.
- `UnmatchedCount / TotalCount` → Eşleşmeme oranı. %5'in üstü clearing gecikmesi veya kayıp anlamına gelir.
- `AmountMismatchCount > 0` → **Her zaman araştırılmalı.** Tutar farkı finansal risk.
- `ProblemCount` trendi artıyorsa → Sistemik sorun var, kök neden araştırması başlatılmalı.

10.5 Ağ Bazlı Karşılaştırma

- Network (BKM/VISA/MSC) bazında aynı KPI'lar. **Hangi ağda daha fazla sorun olduğunu karşılaştırmak için kullanılır.**
- Bir ağda `ProblemCount` yüksek, diğerlerinde düşükse → sorun o ağın dosya formatında veya veri kalitesinde.

10.6 Dosya Bazlı Analiz

- Dosya bazında KPI'lar. **ProblemCount'a göre azalan sırada** sıralıdır — en sorunlu dosya en üsttedir.

- Tek bir dosyada çok yüksek `ProblemCount` → o dosya bozuk olabilir, kaynak sistem bilgilendirilmeli.
- Birden fazla dosyada yaygın `ProblemCount` → sistemik sorun.

---

11. Status ve State Geçiş Analizi

11.1 Dosya Satır Mutabakat Durumu (ReconciliationStatus)

```
[Satır parse edildi]
       |
       ▼
    ┌─────┐
    |Ready| ── Korelasyon anahtarı başarılı oluşturuldu
    └─┬───┘
       | Evaluate claim
       ▼
  ┌──────────┐
  |Processing | ── Bir worker tarafından claim edildi
  └──────────┘
       |
   ┌───┴────┐
   ▼        ▼
┌───────┐ ┌──────┐
|Success| |Failed|
└───────┘ └──────┘
```

Ready (1)

- **Teknik anlam:** Satır parse edildi, korelasyon anahtarı atandı, değerlendirme için hazır.
- **İş anlamı:** Bu satır henüz mutabakat sürecine girmemiş, kuyrukta bekliyor.
- **Ne zaman oluşur:** Dosya ingestion sırasında, korelasyon anahtarı başarılı atandığında.
- **Bu değeri görürsem ne anlamalıyım:** Evaluate henüz çalışmamış veya bu satıra henüz sıra gelmemiş.
- **Sonraki olası durum:** Processing (evaluate claim sırasında).
- **Aksiyon gerekir mi:** Evet — Evaluate tetiklenmelidir.

Processing (4)

- **Teknik anlam:** Bir worker bu satırı claim etmiş, aktif olarak değerlendiriliyor.
- **İş anlamı:** İşlem sürüyor, bekle.
- **Ne zaman oluşur:** Evaluate claim mekanizması sırasında.
- **Bu değeri görürsem ne anlamalıyım:** Normal akışta geçici durum. Eğer uzun süredir Processing'teyse worker çökmüş olabilir.
- **Sonraki olası durum:** Success veya Failed.
- **Aksiyon gerekir mi:** Hayır — `ClaimTimeoutSeconds` (varsayılan 30 dk) aşılırsa otomatik devralınır.

Success (3)

- **Teknik anlam:** Değerlendirme tamamlandı, operasyonlar oluşturuldu.
- **İş anlamı:** Bu satır için mutabakat planı hazır, Execute aşamasına geçilmiştir.
- **Bu değeri görürsem ne anlamalıyım:** Normal beklenen durum. Operasyonlar oluşturulmuş, Execute çalıştırılmalı.
- **Sonraki olası durum:** Terminal — değişmez. (Operasyonlar ayrı entity'de takip edilir.)
- **Aksiyon gerekir mi:** Evet — Execute tetiklenmelidir.

Failed (2)

- **Teknik anlam:** Değerlendirme başarısız oldu veya korelasyon anahtarı oluşturulamadı.
- **İş anlamı:** Bu satır için mutabakat yapılamadı. Manuel müdahale gerekebilir.

- **Bu değeri görürsem ne anlamalıyım:** Ya parse sırasında korelasyon anahtarı üretilemedi (OceanTxnGuid yok, fallback alanlar da eksik) ya da evaluate sırasında exception oluştu.
- **Sonraki olası durum:** Terminal — yeniden değerlendirilmez (dosya tekrar yüklenmedikçe).
- **Aksiyon gerekir mi:** Evet — Alert kontrol edilmeli, kök neden araştırılmalı.

11.2 Operasyon Durumu (OperationStatus)

```
[Evaluate operasyon oluşturur]
        |
        ▼
  ┌─────────┐      ┌────────┐
  |Planned  |──────►|Blocked| ── Önceki operasyon henüz bitmedi
  └─────────┘      └────────┘
      |               | Parent tamamlandı
      |               ▼
      └──────►┌──────────┐
              |Executing | ── Worker claim etti
              └──────────┘
                   |
        ┌──────────┼──────────┐
        ▼          ▼          ▼
  ┌──────────┐ ┌──────┐ ┌─────────┐
  |Completed | |Failed| |Cancelled|
  └──────────┘ └──────┘ └─────────┘
```

Planned (0)

- **Teknik anlam:** Operasyon oluşturuldu, yürütülmeye hazır. Sadece **ilk operasyon** (SequenceIndex=0) bu durumda başlar.
- **İş anlamı:** Kuyrukta bekliyor.
- **Bu değeri görürsem ne anlamalıyım:** Execute henüz bu operasyona ulaşmamış veya retry sonrası yeniden planlanmış.
- **Sonraki olası durum:** Executing (claim edildiğinde).
- **Aksiyon gerekir mi:** Hayır — Execute çalıştırıldığında otomatik işlenecek.

Blocked (1)

- **Teknik anlam:** Önceki operasyon henüz tamamlanmadı veya manuel karar bekliyor.
- **İş anlamı:** Bu operasyon sırası gelmedi.
- **Ne zaman oluşur:** SequenceIndex > 0 olan tüm operasyonlar başlangıçta `Blocked` olur. Manuel gate Pending ise operasyon `Blocked` 'a döner.
- **Bu değeri görürsem ne anlamalıyım:** Normal — önceki adım bekleniyor. Çok uzun süredir Blocked ise önceki operasyon stuck olmuş olabilir.
- **Sonraki olası durum:** Parent `Completed` olduğunda Executing'e promote edilir. Veya Cancelled (branch iptal).
- **Aksiyon gerekir mi:** Hayır — otomatik promote mekanizması var. Manuel gate ise Pending Reviews kontrol edilmeli.

Executing (2)

- **Teknik anlam:** Bir worker bu operasyonu claim etti ve yürütüyor.

- **İş anlamı:** İşlem aktif.
- **Bu değeri görürsem ne anlamalıyım:** Normal geçici durum. Uzun süredir Executing ise lease süresi kontrol edilmeli.
- **Sonraki olası durum:** Completed, Failed, veya Blocked (manuel gate pending).
- **Aksiyon gerekir mi:** Hayır — lease süresi dolunca otomatik devralınır.

Completed (3)

- **Teknik anlam:** Operasyon başarıyla yürütüldü.
- **İş anlamı:** Bu adım tamam, sonraki adım tetiklenebilir.
- **Bu değeri görürsem ne anlamalıyım:** Beklenen final durum. Sorunsuz.
- **Sonraki olası durum:** Terminal — geri dönüş yok.
- **Aksiyon gerekir mi:** Hayır.

Failed (4)

- **Teknik anlam:** Tüm retry denemeleri tükendi.
- **İş anlamı:** Kalıcı hata. Manuel müdahale gerekir.
- **Bu değeri görürsem ne anlamalıyım:** Dış sistem entegrasyonunda kalıcı bir sorun var. Alert oluşturulmuştur.
- **Sonraki olası durum:** Terminal.
- **Aksiyon gerekir mi:** Evet — alert incelenmeli, dış sistem kontrol edilmeli, gerekirse operasyon verileri düzeltilip yeni evaluate tetiklenmeli.

Cancelled (5)

- **Teknik anlam:** Manuel karar sonucu veya expiration sonucu iptal edildi.
- **İş anlamı:** Bu operasyon artık yürütülmeyecek.
- **Ne zaman oluşur:** Manual gate rejected → approve dalı cancelled. Expiration → cancel kararı. `CancelRemaining` akışı.
- **Bu değeri görürsem ne anlamalıyım:** Bilinçli bir karar sonucu iptal edilmiş. Beklenen davranış.
- **Sonraki olası durum:** Terminal.
- **Aksiyon gerekir mi:** Hayır — kararın doğruluğu teyit edilebilir.

11.3 Review Kararı (ReviewDecision)

```
[Review oluşturulur]
        |
        ▼
   ┌─────────┐
   |Pending  |
   └────┬────┘
        |
   ┌────┼──────────┐
   ▼    ▼          ▼
┌────────┐  ┌────────┐  ┌─────────┐
|Approved|  |Rejected|  |Cancelled|
└────────┘  └────────┘  └─────────┘
```

Pending (0)

- **Teknik anlam:** Karar bekliyor.
- **İş anlamı:** Operatör approve veya reject çağrısı yapmalı.
- **Bu değeri görürsem ne anlamalıyım:** İnsan kararı bekleniyor. `ExpiresAt` kontrolü yapılmalı.
- **Sonraki olası durum:** Approved, Rejected, Cancelled (expiration ile).
- **Aksiyon gerekir mi:** Evet — Approve veya Reject çağrısı yapılmalı.

- **Aksiyon gerekir mi:** Evet — Approve veya Reject çağrısı yapılmalı.

Approved (1)

- **Teknik anlam:** Onaylandı.
- **İş anlamı:** Approve dalı operasyonları `Blocked` → çalıştırılmaya hazır promote olur.
- **Bu değeri görürsem ne anlamalıyım:** Karar verilmiş. Execute çalıştırıldığında approve dalı yürütülecek.
- **Aksiyon gerekir mi:** Hayır.

Rejected (2)

- **Teknik anlam:** Reddedildi.
- **İş anlamı:** Reject dalı aktifleşir, approve dalındaki operasyonlar iptal edilir.
- **Bu değeri görürsem ne anlamalıyım:** Karar verilmiş. Execute çalıştırıldığında reject dalı yürütülecek.
- **Aksiyon gerekir mi:** Hayır.

Cancelled (3)

- **Teknik anlam:** Süre doldu ve `ExpirationAction = Cancel`.
- **İş anlamı:** Tüm dallar iptal. `ExpirationFlowAction` ayarına göre kalan akış da iptal olabilir.
- **Bu değeri görürsem ne anlamalıyım:** Süresinde karar verilmemiş, otomatik iptal uygulanmış.
- **Aksiyon gerekir mi:** Duruma göre — iptal kararının etkisi değerlendirilmeli.

11.4 Retry ve Backoff Mekanizması

Başarısız operasyonda retry planlaması:

- `RetryCount` artırılır
- `RetryCount >= MaxRetries` → **Failed** (terminal)
- `RetryCount < MaxRetries` → **Planned** + `NextAttemptAt = Şimdi + 30 * 2^RetryCount saniye`
  - a. retry: 60 saniye sonra
  - b. retry: 120 saniye sonra
  - c. retry: 240 saniye sonra (4 dakika)
  - d. retry: 480 saniye sonra (8 dakika)
  - e. retry: 960 saniye sonra (16 dakika)

---

12. İş Kuralları Özeti

Dosya Bütünlüğü Kuralları

1. `ExpectedCount != TotalCount` → dosya `Failed` olarak finalize edilir, mesajda uyarı eklenir.
2. `ErrorCount > 0` → dosya `Failed`, mesajda kaç satırın hatalı olduğu belirtilir.
3. `IsArchived = false` → dosya `Failed`, arşiv hatası mesajı eklenir.
4. Bu üç koşulun hepsinin temiz olması gerekir → `Status = Success`.

Korelasyon Anahtarı Kuralları

1. **Öncelik 1:** `OceanTxnGuid > 0` → CorrelationKey = `OceanTxnGuid`, CorrelationValue = OceanTxnGuid değeri. `ReconciliationStatus = Ready`.
2. **Öncelik 2 (fallback):** `Rrn:CardNo:ProvisionCode:Arn:Mcc:Amount:Currency` birleşimi. Tüm alanlar doluysa `Ready`; herhangi biri boşsa `Failed`.

3. **Kart duplikat key:** OceanTxnGuid.
4. **Clearing duplikat key:** `ClrNo:ControlStat`.

Duplikat Tespiti Kuralları

1. **Kart dosyalarında:** `OceanTxnGuid` aynı olan satırlar duplikat. İlk gelen `Primary`, sonrakiler `Secondary`.
2. **Clearing dosyalarında:** `ClrNo:ControlStat` aynı olan satırlar duplikat.
3. **Duplikat yoksa:** `Unique`.
4. **Çakışma varsa (aynı key, farklı içerik):** `Conflict`.

Clearing Eşleştirme Kuralları

1. Korelasyon anahtarı (öncelikle `OceanTxnGuid`) üzerinden eşleşme yapılır.
2. Clearing dosyası yüklendiğinde → mevcut kart satırlarına `matched_clearing_line_id` atanır.
3. Kart dosyası yüklendiğinde → mevcut clearing satırlarından en son oluşturulan eşleştirilir.
4. Eşleşme bulunamazsa → `match_status = UNMATCHED_CARD` (view tarafında).

Mismatch Hesaplama Kuralları (SQL View'da)

1. **Tutar uyuşmazlığı:** `ABS(card_original_amount - clearing_source_amount) > 0.01` → **0.01 toleransı var.** Bu, kur farkı veya yuvarlama farkını görmezden gelmek için. 0.01'den büyük fark gerçek bir uyumsuzluğa işaret eder.
2. **Para birimi uyuşmazlığı:** `card_original_currency IS DISTINCT FROM clearing_source_currency` → Birebir eşleşme beklenir.
3. **Tarih uyuşmazlığı:** `card_transaction_date::text IS DISTINCT FROM clearing_txn_date::text` → String olarak karşılaştırılır.
4. **Durum uyuşmazlığı:** Kart `Successful` + Clearing `control_stat != Normal` **VEYA** Kart `Unsuccessful` + Clearing `control_stat = Normal` → Çelişki.

Arşiv Uygunluk Kuralları

1. Dosya, satır, evaluation, operation, review, execution, alert statülerinin **tamamı terminal durumda** olmalı.
2. Dosyanın son güncellenmesinden bu yana **MinLastUpdateAgeHours** (varsayılan 72 saat) geçmiş olmalı.
3. Terminal statüler konfigürasyonla belirlenir:
   - IngestionFile: `Success`, `Failed`
   - IngestionFileLine: `Success`, `Failed`
   - ReconciliationEvaluation: `Completed`, `Failed`
   - ReconciliationOperation: `Completed`, `Failed`, `Cancelled`
   - ReconciliationReview: `Approved`, `Rejected`, `Cancelled`
   - ReconciliationOperationExecution: `Completed`, `Failed`, `Skipped`
   - ReconciliationAlert: `Consumed`, `Failed`, `Ignored`

---

13. Veri Akışları

Ana Akış

```
[Dosya Kaynağı (SFTP/Lokal)]
          |
          ▼
```

```
┌──────────────────────┐
|  FileIngestion       | → ingestion.file + ingestion.file_line + ingestion.card_*_detail /
clearing_*_detail
|  POST /FileIngest    | → Duplikat tespiti + Clearing eşleştirmesi
└──────────┬───────────┘
         | Satırlar ReconciliationStatus = Ready
         ▼

┌──────────────────────┐
|  Evaluate            | → reconciliation.evaluation + reconciliation.operation + reconciliation.review
|  POST /Evaluate      | → Satırlar ReconciliationStatus = Success (veya Failed)
└──────────┬───────────┘
         | Operasyonlar Planned/Blocked
         ▼
┌──────────────────────┐
|  Execute             | → reconciliation.operation_execution + reconciliation.alert
|  POST /Execute       | → Operasyonlar Completed/Failed/Cancelled
└──────────┬───────────┘
         |
    ┌────┴────┐
    ▼         ▼
┌─────────┐ ┌──────────────┐
| Arşiv   | | Manuel Onay  |
| POST    | | POST Approve |
| /Run    | | POST Reject  |
└─────────┘ └──────────────┘
```

Raporlama Akışı

```
[ingestion.file + file_line + card_*_detail + clearing_*_detail]
         |
         ▼
[reporting.vw_base_card_transaction]  ← BKM+VISA+MSC normalize UNION
[reporting.vw_base_clearing_transaction] ← BKM+VISA+MSC normalize UNION
         |
         ▼ LEFT JOIN (card.matched_clearing_line_id = clearing.file_line_id)
[reporting.vw_reconciliation_matched_pair]  ← Mismatch flag'leri burada hesaplanır
         |
    ┌────┴──────┬──────────┬──────────┐
    ▼    ▼      ▼          ▼          ▼
 unmatched  amount_   status_   problem_   clean_
 _card     mismatch  mismatch  records    matched
    |
    ▼
[summary_daily / summary_by_network / summary_by_file / summary_overall]
```

---

14. SQL View Okuma Rehberi

View Katman Yapısı

Katman	View	Amacı
Temel	`vw_base_card_transaction`	BKM/VISA/MSC kart detaylarını normalize eder
Temel	`vw_base_clearing_transaction`	BKM/VISA/MSC clearing detaylarını normalize eder
Eşleştirme	`vw_reconciliation_matched_pair`	Kart ve clearing'i yan yana getirir, mismatch flag'leri hesaplar
İş	`vw_reconciliation_unmatched_card`	Sadece eşleşmeyenler
İş	`vw_reconciliation_amount_mismatch`	Sadece tutar uyuşmazlığı

İş	`vw_reconciliation_amount_mismatch`	Sadece tutar uyuşmazlığı
İş	`vw_reconciliation_status_mismatch`	Sadece durum çelişkisi
İş	`vw_reconciliation_clean_matched`	Tam temiz kayıtlar
İş	`vw_reconciliation_problem_records`	Herhangi bir sorunu olan tüm kayıtlar
Özet	`vw_reconciliation_summary_daily`	Günlük KPI'lar
Özet	`vw_reconciliation_summary_by_network`	Ağ bazlı KPI'lar
Özet	`vw_reconciliation_summary_by_file`	Dosya bazlı KPI'lar
Özet	`vw_reconciliation_summary_overall`	Tek satırlık genel durum
Operasyonel	`vw_file_ingestion_summary`	Dosya yükleme özeti + success_rate + has_count_mismatch
Operasyonel	`vw_reconciliation_file_summary`	Dosya bazlı mutabakat durumu (ready/processing/success/failed dağılımı)
Operasyonel	`vw_reconciliation_line_detail`	Satır detay analizi (evaluation/operation aggregates, age_days)
Operasyonel	`vw_reconciliation_operation_tracker`	Operasyon takibi (lease, retry, execution bilgileri)
Operasyonel	`vw_reconciliation_pending_actions`	Bekleyen aksiyonlar + review süre/karar bilgisi
Operasyonel	`vw_reconciliation_alert_dashboard`	Uyarı panosu (severity, type, age_hours)
Operasyonel	`vw_daily_reconciliation_summary`	Günlük dosya bazlı özet (content_type, file_type kırılımı)
Operasyonel	`vw_reconciliation_aging`	Yaşlanma analizi (0-1/2-3/4-7/8-14/15-30/30+ gün bucket'ları)
Operasyonel	`vw_archive_audit_trail`	Arşiv denetim izi (days_to_archive hesaplaması)
Operasyonel	`vw_clearing_dispute_monitor`	Clearing dispute izleme (control_stat != Normal VEYA dispute_code != None)

Mismatch Flag Hesaplama Detayı

`has_amount_mismatch`

```
ABS(COALESCE(c.original_amount, 0) - COALESCE(clr.source_amount, 0)) > 0.01
```

- **0.01 toleransı:** Kur dönüşümü ve yuvarlama farklarını görmezden gelmek için.
- **COALESCE:** Null değerler 0 olarak işlenir. Bir taraf null ise → fark tüm tutar kadar.
- **Bu değer TRUE ise:** Kart ve clearing tutarları arasında 1 kuruştan fazla fark var. Bu **gerçek bir uyumsuzluk**.
- **Eşleşmemiş kayıtlarda:** `NULL` döner (clearing yoksa hesaplanamaz).

`has_currency_mismatch`

```
c.original_currency IS DISTINCT FROM clr.source_currency
```

- **IS DISTINCT FROM:** NULL-safe karşılaştırma. Biri null, diğeri değilse → TRUE.
- **Bu değer TRUE ise:** Kart ve clearing farklı para birimi raporluyor. **Kritik uyarı** — ya veri hatası, ya da çok para birimli işlem.

`has_date_mismatch`

```
c.transaction_date::text IS DISTINCT FROM clr.txn_date::text
```

- **String karşılaştırma:** Integer tarihler text'e dönüştürülüp karşılaştırılır.
- **Bu değer TRUE ise:** İşlem tarihleri farklı. Genellikle gün sonu kesim saati farklılığından kaynaklanır.

`has_status_mismatch`

```
WHEN c.is_successful_txn = 'Successful' AND COALESCE(clr.control_stat, 'Normal') <> 'Normal' THEN TRUE
WHEN COALESCE(c.is_successful_txn, 'Unsuccessful') = 'Unsuccessful' AND COALESCE(clr.control_stat,
'Normal') = 'Normal' THEN TRUE
```

- **Çelişki 1:** Kart "başarılı" diyor ama clearing "normal değil" (dispute, chargeback vb.)
- **Çelişki 2:** Kart "başarısız" diyor ama clearing "normal" diyor.
- **Bu değer TRUE ise:** İki sistem arasında işlem sonucu tutarsız. **Mutlaka araştırılmalı.**

Problem Records vs Clean Matched Farkı

Problem Records (herhangi biri doğruysa kayıt dahil):

- `match_status = 'UNMATCHED_CARD'`
- `has_amount_mismatch = TRUE`
- `has_currency_mismatch = TRUE`
- `has_date_mismatch = TRUE`
- `has_status_mismatch = TRUE`
- `COALESCE(duplicate_status, 'Unique') <> 'Unique'`

Clean Matched (tümü sağlanmalı):

- `match_status = 'MATCHED'`
- `has_amount_mismatch = FALSE`
- `has_currency_mismatch = FALSE`
- `has_date_mismatch = FALSE`
- `has_status_mismatch = FALSE`

Dikkat: Clean tanımı `duplicate_status` u kontrol etmez — duplikat Primary bile olsa tutar/tarih/durum temizse "clean" sayılır. Ama Problem tanımı duplikat'ı dahil eder. Bu, `TotalCount != ProblemCount + CleanCount` olabileceği anlamına gelir — arada duplikat Primary olan ama mismatch olmayan kayıtlar olabilir.

Aging Analizi Yorumlama

- **0-1 Gün:** Normal, yeni yüklemeler.
- **2-3 Gün:** Kabul edilebilir, clearing gecikmesi olabilir.
- **4-7 Gün:** Dikkat gerektiren birikim. Evaluate/Execute planlaması gözden geçirilmeli.
- **8-14 Gün:** Sorunlu. Ya evaluate çalışmıyor ya da kalıcı hatalar var.
- **15-30 Gün:** Kritik. Acil müdahale.
- **30+ Gün:** Muhtemelen manuel müdahale gerektiren kayıtlar.

---

15. Rapor Yorumlama Kılavuzu

Günlük Özet Yorumlama

Metrik	Normal Aralık	Uyarı Eşiği	Kritik Eşik	Aksiyon
`CleanCount / TotalCount`	> %98	< %95	< %90	Kök neden araştırması
`UnmatchedCount / TotalCount`	< %2	> %5	> %10	Clearing dosyası gecikmesi kontrol et
`AmountMismatchCount`	0	> 10	> 100	Her kayıt tek tek araştırılmalı
`CurrencyMismatchCount`	0	> 0	> 5	Kur dönüşüm mantığı gözden geçirilmeli
`StatusMismatchCount`	0	> 5	> 50	Dispute süreci kontrol edilmeli
`ProblemCount` trendi	Azalan	Sabit	Artan	Sistemik sorun aranmalı

Ağ Bazlı Karşılaştırma

- Bir ağda `ProblemCount` oranı diğerlerinden belirgin şekilde yüksekse → **o ağın dosya kalitesi veya parse kuralları sorunlu.**
- BKM genellikle yerel işlemleri içerir — düşük uyumsuzluk beklenir.
- VISA/MSC uluslararası işlemleri içerir — kur farkı kaynaklı tutar uyumsuzlukları daha olasıdır.

Dosya Bazlı Analiz

- Dosya bazında KPI'lar. **ProblemCount'a göre azalan sırada** sıralıdır — en sorunlu dosya en üsttedir.
- Tek bir dosyada çok yüksek `ProblemCount` → o dosya bozuk olabilir, kaynak sistem bilgilendirilmeli.
- Birden fazla dosyada yaygın `ProblemCount` → sistemik sorun.

16. Analist Kılavuzu

Günlük Kontrol Listesi

1. **`GET /Summary`** çağır → `CleanCount / TotalCount` oranına bak. %98 altıysa alarm.
2. **`GET /Summary/Daily`** çağır → son 7 günlük trende bak. `ProblemCount` artıyorsa kök neden ara.
3. **`GET /Problems`** çağır → `HasAmountMismatch = true` olanları filtrele. Her biri finansal risk.
4. **`GET /Unmatched`** çağır → eski tarihlilere odaklan. 3+ gün eşleşmemiş kayıtlar sorunlu.
5. **`GET /Reviews/Pending`** çağır → `ExpiresAt` yakın olanları öncelikle çöz.
6. **`GET /Alerts`** çağır → `Severity = Error` olanları incele.

Karar Ağacı

```
AmountDifference > 0 ve küçük (< 1.00)?
  → Muhtemelen kur/yuvarlama farkı. İzle ama acil değil.

AmountDifference > 100?
  → Yüksek tutarlı uyumsuzluk. Acil araştırma gerekir.

AmountDifference < 0 (negatif)?
```

```
  → Clearing tutarı kart tutarından yüksek. Olağandışı. Araştır.

MatchStatus = UNMATCHED_CARD ve kayıt 1 günden eski?
  → Normal, clearing dosyası bekleniyor.

MatchStatus = UNMATCHED_CARD ve kayıt 3-7 günden eski?
  → Uyarı: clearing dosyası gecikmiş olabilir. Kaynak kontrol edilmeli.

MatchStatus = UNMATCHED_CARD ve kayıt 7+ günden eski?
  → Kritik: Clearing dosyası gelmemiş veya korelasyon sorunu. Escalate.

HasStatusMismatch = true?
  → Dispute/chargeback olabilir. vw_clearing_dispute_monitor kontrol et.

DuplicateStatus = Conflict?
  → Aynı işlem farklı verilerle gelmiş. Veri kalitesi sorunu. Kaynak incelenmeli.

DuplicateStatus = Secondary?
  → Bu bir duplikat kopya. İşlemsel olarak Primary kaydı takip et.

HasCurrencyMismatch = true?
  → Kritik. Farklı para birimleri arasında karşılaştırma yapılmış. Veri hatası veya çok para birimli
işlem.
```

Haftalık Derinlik Analizi

1. **Ağ bazlı karşılaştırma:** `GET /Summary/Network` → hangi ağda anomali var?
2. **Dosya bazlı analiz:** `GET /Summary/File` → en sorunlu 5 dosyayı incele.
3. **Aging kontrolü:** `vw_reconciliation_aging` view'ından 15+ gün bucket'ını sorgula.
4. **Archive audit:** `vw_archive_audit_trail` → `days_to_archive` ortalamasını hesapla. Artıyorsa arşiv süreci yavaşlamış.

---

17. Geliştirici Notları

Claim Mekanizması (Evaluate)

- `SERIALIZABLE` izolasyon seviyesinde transaction kullanılır. Bu, concurrent worker'lar arasında aynı satırın iki kez claim edilmesini önler.
- Claim marker (`EVAL_CLAIM:{GUID}`) ile doğrulama yapılır. Başka worker'ın aynı anda claim ettiği durumda claim sayısı 0 döner ve boş liste ile çıkılır.
- Stale claim tespiti: `UpdateDate <= şimdi - ClaimTimeoutSeconds` olan Processing satırlar yeniden claim edilebilir.
- Batch persistans sırasında hata alınırsa fallback olarak tek tek persistans denenir (batch başarısız → one-by-one). Zaten persist edilmiş kayıtlar `alreadyPersisted` kontrolüyle atlanır.

Lease Mekanizması (Execute)

- `ExecuteUpdateAsync` ile atomik claim yapılır. `LeaseOwner` = `exec:{MachineName}:{ProcessId}` .
- Lease süresi dolmuş operasyonlar başka worker tarafından devralınabilir.
- Claim başarısızsa (0 row updated) → döngüde continue, başka operasyon denenir.
- Lease release: Operasyon tamamlandığında, bloke olduğunda veya retry'a girdiğinde `LeaseExpiresAt` ve `LeaseOwner` null'a çekilir.

İdempotency

- `IdempotencyKey` formatı:
  - Temel: `{OperationCode}:{FileLineId}:{SequenceIndex}`
  - CorrelationValue varsa: `...:{CorrelationValue}:{DifferenceAmount}` veya `...:{OriginalTransactionId}` veya `...:{CorrelationValue}:{Branch}`
  - CorrelationValue yoksa: `...:{Branch}`
- Aynı key ile `Completed` operasyon varsa → yeni operasyon `Skipped` olarak tamamlanır.
- Bu, aynı evaluate sonucunun tekrar execute edilmesinde çift işlemi önler.

Otomatik Arşiv (Fire-and-Forget)

- Execute handler'da `Task.Run` ile arka planda başlatılır.
- Yeni DI scope oluşturulur (`_serviceScopeFactory.CreateScope()`).
- `CancellationToken.None` ile çalışır — ana request iptal edilse bile arşiv devam eder.
- Hata durumunda sadece log yazılır, ana response'u etkilemez.
- **Tetikleme koşulu:** `ArchiveOptions.AutoArchiveAfterExecute == true` VE `response.TotalSucceeded > 0` .

Paralel Dosya İşleme

- `EnableParallelProcessing = true` ve birden fazla dosya varsa, dosyalar `SemaphoreSlim` ile kontrollü paralellikte işlenir.
- Her paralel iş kendi DI scope'unu kullanır (scope'lu `FileIngestionOrchestrator` çözümlenir).
- Tek dosya ise paralel mod aktif olsa bile sıralı çalışır.

Bulk Insert

- PostgreSQL'de `COPY ... FROM STDIN (FORMAT BINARY)` kullanılır.
- SQL Server'da `SqlBulkCopy` kullanılır.
- Enum değerleri string olarak yazılır.
- Konfigürasyondaki `UseBulkInsert` flag'i ile açılıp kapatılabilir. `false` ise standart `AddRangeAsync` + `SaveChangesAsync` .
- Typed detail entity'ler (card_visa_detail, clearing_msc_detail vb.) bulk insert sonrasında ayrı persist edilir.

Hata Yönetimi Stratejisi

- Tüm handler'lar try-catch sarmalı içindedir. Exception asla çıplak fırlatılmaz.
- Hata durumunda error mapper ile yapılandırılmış `ErrorDetail` nesnesi oluşturulur (Code, Message, Step, Severity).
- Handler seviyesinde yakalanan exception'lar loglama + hata response olarak döner.
- Service seviyesindeki hatalar errors listesine eklenir ve response ile birlikte döner.
- **Hiçbir endpoint HTTP 500 dönmez** — hata bilgisi response body'sinde yapılandırılmış şekilde iletilir.

Recovery Mekanizması (File Ingestion)

Aynı dosya tekrar gönderildiğinde (`FileKey` eşleştiğinde):

1. **`RequiresFullRecovery`** : Dosya `Processing` durumunda kalmışsa → eksik satırlar tamamlanır, hatalı satırlar retry edilir.
2. **`RequiresArchiveOnlyRecovery`** : Dosya parse edilmiş ama arşivlenmemişse → sadece arşiv akışı çalışır.
3. **`RequiresReArchive`** : Dosya arşivlenmiş ama sonradan satır güncellenmiş → yeniden arşivlenir.
4. **Tam duplikat:** Dosya başarılı ve değişiklik yoksa → mevcut kayıt döner.

---

18. Senaryo Bazlı Davranışlar

Senaryo 1: Normal Gün Sonu Mutabakat Akışı

1. **18:00** — BKM kart dosyası SFTP'den çekilir:
   - `POST /FileIngestion` → FileSourceType=Remote, FileType=Card, FileContentType=Bkm
   - Sonuç: 50.000 satır, Status=Success, tümü ReconciliationStatus=Ready
2. **18:30** — BKM clearing dosyası yüklenir:
   - `POST /FileIngestion` → FileType=Clearing, FileContentType=Bkm
   - Sonuç: 49.800 satır. Clearing eşleştirmesi otomatik çalışır, 49.500 kart satırı eşleşir.
3. **19:00** — Evaluate tetiklenir:
   - `POST /Reconciliation/Evaluate` → boş request (tüm Ready satırlar)
   - Sonuç: 50.000 satır değerlendirilir. 200 tanesi unmatched, 50 tanesi tutar uyuşmazlığı, 5 tanesi durum çelişkisi.
   - CreatedOperationsCount: 255 operasyon oluşturulur.
4. **19:05** — Execute tetiklenir:
   - `POST /Reconciliation/Operations/Execute` → boş request
   - 250 otomatik operasyon tamamlanır, 5 tanesi manuel inceleme bekliyor.
   - TotalSucceeded: 250, TotalFailed: 0
5. **19:10** — Operatör pending reviews'ı kontrol eder:
   - `GET /Reviews/Pending` → 5 kayıt
   - 3 tanesini approve eder, 2 tanesini reject eder.
6. **19:15** — Execute tekrar tetiklenir:
   - Approve dalları çalışır, reject dalları çalışır.
   - TotalSucceeded: 8 (3 approve gate + 3 approve branch + 2 reject branch)
7. **20:00** — Günlük rapor kontrol:
   - `GET /Summary/Daily` → CleanCount: 49.500, ProblemCount: 500, TotalCount: 50.000
   - Mutabakat başarı oranı: %99

Senaryo 2: Clearing Dosyası Gecikmiş

1. Kart dosyası yüklendi (50.000 satır), evaluate çalıştı.
2. Clearing dosyası 3 gün gecikti.
3. **3 gün boyunca:** `GET /Unmatched` → 50.000 kayıt (tümü eşleşmemiş)
4. **3. gün clearing gelir:** `POST /FileIngestion` → Clearing yüklenir
5. Otomatik eşleştirme çalışır → 49.900 satır eşleşir
6. **Ancak:** Zaten evaluate edilmiş satırlar için yeni evaluate gerekmez. Eşleştirme sadece raporlama view'larını etkiler.
7. Reporting view'larda `MatchStatus` güncellenir, `GET /Summary` yeniden sorgulandığında `UnmatchedCount` düşer.

Senaryo 3: Manuel Onay Süresi Dolmuş

1. Evaluate sırasında `CreateManualReview` operasyonu oluşturuldu

1. Evaluate sırasında `CreateHandlerReview` operasyonu oluşturuldu.
2. Review `ExpiresAt = 48 saat sonra`, `ExpirationAction = Reject`, `ExpirationFlowAction = CancelRemaining`
3. 48 saat geçti, kimse karar vermedi.
4. Execute tetiklendiğinde:
   - Review süre dolmuş → otomatik `Rejected` kararı uygulanır
   - Gate operasyonu `Completed` olur
   - Reject dalı aktifleşir (`Blocked` → promote → `Executing`)
   - Approve dalı `Cancelled` olur
   - `ExpirationFlowAction = CancelRemaining` olduğu için gate'ten sonraki **tüm kalan operasyonlar da Cancelled** olur
5. Sonuç: Sadece reject dalı yürütülür, geri kalan tüm akış iptal.

Senaryo 4: Worker Çökmesi ve Recovery

1. Evaluate sırasında Worker-A 10.000 satırı claim etti (Processing + claim marker).
2. Worker-A çöktü (process kill).
3. 30 dakika geçti (ClaimTimeoutSeconds).
4. Worker-B Evaluate çalıştırdığında:
   - `ReconciliationStatus = Processing AND UpdateDate <= şimdi - 1800s` sorgusu çalışır
   - Stale satırlar bulunur
   - Worker-B bu satırları **yeni claim marker ile** yeniden claim eder
   - Değerlendirme normal şekilde devam eder
5. Hiçbir veri kaybı olmaz. Çift işleme olmaz (claim marker doğrulaması).

Senaryo 5: Aynı Dosya Tekrar Gönderilir

Durum A — Dosya başarılı:

- İkinci gönderim: FileKey eşleşir → mevcut kayıt döner, mesaj "Duplicate file received" güncellenir.

Durum B — Dosya Processing'te kalmış (worker çökmüş):

- İkinci gönderim: `RequiresFullRecovery` tetiklenir
- Eksik satırlar `LastProcessedByteOffset`'ten itibaren tamamlanır
- Hatalı satırlar yeniden parse edilmeye çalışılır (max retry count'a kadar)
- Arşiv kontrolü yapılır
- Dosya finalize edilir

Durum C — Dosya parse edilmiş ama arşivlenmemiş:

- İkinci gönderim: `RequiresArchiveOnlyRecovery` tetiklenir
- 3 deneme ile arşiv yeniden denenir
- Başarılıysa `IsArchived = true` ve dosya finalize
- 3 denemede de başarısızsa `IsArchived = false` kalır, dosya `Failed`

Senaryo 6: Tutar Uyuşmazlığı Detay Analizi

1. `GET /Transactions?HasAmountMismatch=true` ile uyuşmazlıklar listelenir.
2. **Kayıt A:**
   - `CardOriginalAmount = 100.00`, `ClearingSourceAmount = 99.50`, `AmountDifference = 0.50`
   - **Yorum:** 50 kuruşluk fark. `HasCurrencyMismatch = false` ise kur farkı değil. Yuvarlama veya komisyon farkı

Yorum: Sıfır karşılık farkı `HasCurrencyMismatch = false` ise kur farkı değil. Yuvarlama veya komisyon farkı olabilir. İzlenmeli ama acil değil.

3. **Kayıt B:**
   - `CardOriginalAmount = 500.00`, `ClearingSourceAmount = 0.00`, `AmountDifference = 500.00`
   - **Yorum:** Clearing tutarı sıfır. Bu **büyük olasılıkla veri hatası**. Clearing kaynağı kontrol edilmeli.
4. **Kayıt C:**
   - `CardOriginalAmount = 250.00`, `ClearingSourceAmount = 250.00`, `AmountDifference = 0.00`
   - **Yorum:** `HasAmountMismatch = false` olacağı için bu filtrede görünmez. Tolerans 0.01'e kadar.
5. **Kayıt D:**
   - `CardOriginalAmount = 100.00`, `ClearingSourceAmount = 100.005`, `AmountDifference = -0.005`
   - **Yorum:** `ABS(-0.005) = 0.005 < 0.01` → `HasAmountMismatch = FALSE`. Tolerans içinde, sorun yok.

Senaryo 7: Büyük Hacimli Toplu Çalıştırma

1. 500 dosya birikmiş (1 haftalık gecikme sonrası).
2. `POST /FileIngestion` (boş request, Remote) → tüm dosyalar paralel işlenir (`EnableParallelProcessing = true`)
3. `POST /Evaluate` (boş request) → ChunkSize: 10.000, ClaimRetryCount: 5
4. Evaluate 2.5 milyon satır işler, 50.000 operasyon oluşturur
5. `POST /Execute` (boş request) → MaxEvaluations: 500.000
6. Execute tüm operasyonları yürütür
7. AutoArchiveAfterExecute = true → arka planda arşiv başlar
8. `GET /Summary` → genel durumu gösterir

Senaryo 8: Alert Yorumlama

1. `GET /Alerts?AlertStatus=Pending` → 10 alert
2. **Alert 1:** `AlertType = "EvaluationFailed"`, `OperationId = Guid.Empty`
   - **Yorum:** Evaluate sırasında satır değerlendirilemedi. OperationId boş çünkü operasyon bile oluşturulamadı. Message'da root cause var.
3. **Alert 2:** `AlertType = "OperationExecutionFailed"`, `OperationId = {guid}`
   - **Yorum:** Operasyon tüm retry haklarını tüketmiş. Dış sistem entegrasyonunda kalıcı sorun. OperationId ile operasyon detayına bakılmalı.